Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hf.1'in HTTP metot/durum kodu/`curl` anatomisi aynı `curl` sözdiziminde bu kez kendi sunucunuza gider; Hf.3'ün `interface Etkinlik/Koltuk/Bilet` TS tipleri bu haftanın Pydantic modellerinde alan adı ve anlam olarak birebir karşılığını bulur — yalnız camelCase yerine Python'da snake_case yazılır.

» Ünite 1 — İlk Uygulama, Yol/Sorgu/Gövde Parametreleri

Sanal Ortam (*venv*)

`python -m venv .venv` projeye özel, izole bir Python kurulumu oluşturur; farklı projelerin farklı paket sürümü ihtiyaçları küresel `pip install` ile çakışmaz. Aktivasyon sonrası `pip install -r requirements.txt`.

ASGI Sunucusu: `uvicorn`

FastAPI hangi yola hangi fonksiyonun cevap vereceğini tanımlayan bir ÇERÇEVEdir; TCP bağlantısını dinleyip HTTP'yi konuşan gerçek süreç `uvicorn`dur. `uvicorn app.main:app --reload` — `app.main` modül yolu, ikinci `app` FastAPI nesnesinin adı, `--reload` kaynak değişince otomatik yeniden başlatır.

Yol Parametresi (*Path Parameter*)

`@app.get("/etkinlikler/{etkinlik_id}")` — `{}` içindeki ad URL'nin bir parçasını yakalar; `etkinlik_id: int` tip ipucu FastAPI'nin metni otomatik `int`'e çevirmesini sağlar.

Sorgu ve Gövde Parametresi

Yol şablonunda geçmeyen bir fonksiyon parametresi (`mekan: str | None = None`) otomatik olarak isteğe bağlı bir `?mekan=...` sorgu dizesi sayılır; bir `POST`/`PUT` isteğinin JSON gövdesi ise parametre tipinin bir Pydantic `BaseModel` alt sınıfı olmasını gerektirir (tam tanım Ünite 2).

Ortak Hata / Tuzak

`etkinlik_id: int` yoluna `/abc` göndermek **404 değil** `422 Unprocessable Entity` üretir — yol eşleşiyor, yalnız parametrenin tipi uymuyor.

» Ünite 2 — Pydantic Modelleri ve Doğrulama

`BaseModel` Nedir

Bir Python sınıfının alanlarının tip ipuçlarından otomatik doğrulama/ayrıştırma mantığı kazanması; `tsc` yalnız derleme zamanı denetler, Pydantic ise çalışma zamanında gelen veriyi denetler, uymayanı reddeder.

`Field` Kısıtları

`kapasite: int = Field(gt=0)` — bir alanın yalnız tipi değil değer aralığı da doğrulanır (`gt`/`ge`/`lt`/`le`/`min_length`/`max_length`).

422 Doğrulama Hatası Anatomisi

Cevap gövdesindeki `detail` bir dizidir: `loc` (hangi alan, ör. `["body","kapasite"]`), `msg` (açıklama), `type` (hata türü) — doğrulama başarısızsa işleyici fonksiyon hiç çağrılmaz.

Giriş/Çıkış Modeli Ayrımı

`EtkinlikOlustur` (istemciden gelen, henüz `id`'si olmayan) vs `Etkinlik` (`id` atanmış, sunucunun döndürdüğü tam nesne) — dışarıdan gelen ile dışarıya giden veri şekli aynı olmak zorunda değildir.

camelCase/snake_case Köprüsü

`KamelModel` taban sınıfı `alias_generator=to_camel, populate_by_name=True` ile Python'da snake_case (`bilet_fiyati`), JSON'da camelCase (`biletFiyati`) üretir — Hf.3'ün TS arayüzleri hiç değişmeden bu JSON'la eşleşir.

İpucu

Hf.3'ün `'bos'|'secili'|'dolu'` union'ı Python'da `Literal['bos','secili','dolu']`'dur; `'secili'` yalnız tarayıcıda anlamlıdır — sunucu yalnız `'bos'`/`'dolu'` geçer.

» Ünite 3 — CRUD, Durum Kodları, Hata Cevapları

Bellekte Depo

`dict[int, Etkinlik]` (ve benzerleri) — sunucu süreç kapandığında (`--reload` dahil) tüm veri tamamen kaybolur; kalıcılık Hf.8'in konusudur.

Durum Kodları: `PUT` vs `PATCH`

`POST`→**201**, `GET`→**200**, `DELETE`→**204** (gövdesiz); `PUT` tüm alanları yeniden yazar (`Etkinlik`), `PATCH` yalnız belirtilen alanı değiştirir (`Koltuk`'un `durum`'u) — ikisi de **200** döner.

Koltuk Rezervasyonu İş Kuralı

`POST /biletler` önce `koltuk_id`'nin var olup olmadığını denetler (`404`), sonra `durum`'unun `'dolu'` olup olmadığını kontrol eder (`409`); `'bos'`sa bilet oluşturulur, koltuk `'dolu'`ya çevrilir, `201` döner.

`APIRouter` ile Ayrıştırma

Her varlık (Etkinlik/Koltuk/Bilet) için ayrı bir `APIRouter`; `main.py`'de `include_router(...)` ile birleştirilir — büyük bir `.cpp` dosyasını sorumluluğa göre bölme disiplininin FastAPI karşılığı.

Ortak Hata / Tuzak

`409 Conflict`'i `400 Bad Request` ile karıştırmak: istek format olarak geçerlidir (`422` tetiklenmez), ama sunucudaki mevcut durumla çelişir — veri geçerli, iş kuralı ihlal edilmiştir.

404/409/201 dallarını uygulayan uç nokta (`bilet_olustur` etiketi):

```python
@router.post("", response_model=Bilet, status_code=201)
def bilet_olustur(veri: BiletOlustur, depo: Depo = Depends(get_depo))
      -> Bilet:
   koltuk = depo.koltuk_getir(veri.koltuk_id)
   if koltuk is None:
      raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
   if koltuk.durum == "dolu":
      raise HTTPException(status_code=409, detail="Bu koltuk zaten
            satılmış")
   bilet = depo.bilet_ekle(veri)
   depo.koltuk_guncelle(koltuk.id, "dolu")
   return bilet
```

» Ünite 4 — OpenAPI Belgeleri ve Bağımlılık Enjeksiyonu

`/docs` ve `/openapi.json`

FastAPI'nin yazılan her rotadan otomatik ürettiği interaktif Swagger UI belgesi ve aynı belgenin makine-okunabilir JSON şeması — hiçbir ek kod yazmadan, yalnız tip ipuçlarından türetilir.

`Depends` ile Bağımlılık Enjeksiyonu

`depo: Depo = Depends(get_depo)` — rota fonksiyonu deponun nasıl oluşturulduğuyla ilgilenmez; FastAPI her istekte `get_depo()`'yu çağırıp tüm router'ların kullandığı tek, paylaşılan `Depo`'yu bu parametreye yerleştirir.

`TestClient` ile İlk API Testi

`from fastapi.testclient import TestClient` — gerçek bir sunucu ayağa kaldırmadan Python içinde istek gönderir; `assert response.status_code == 201` ile cevap denetlenir. `app.dependency_overrides[get_depo]` Her teste temiz bir depo verir — yoksa bir testin kaydı bir sonraki testi etkiler.

» Komutlar

`python -m venv .venv`	İzole bir sanal ortam oluşturur
`pip install -r requirements.txt`	Bağımlılıkları sanal ortama kurar
`uvicorn app.main:app --reload`	Sunucuyu 8000 portunda çalıştırır, kod değişince yeniden başlar
`python -m pytest -q`	Tüm testleri sessiz (özet) modda çalıştırır
`curl -i http://127.0.0.1:8000/etkinlikler`	Etkinlik listesini başlıklarıyla ister

5 • İzleme **Nerede Kullanılır:** Yol dışı ek bilgi → sorgu parametresi · URL'nin bir parçası → yol parametresi · JSON gövdesi gerekiyor → Pydantic gövde parametresi · Kayıt yok → 404 · Veri geçersiz → 422 · Format doğru ama durumla çelişiyor → 409 · tüm alanları değiştir → `PUT` · tek alanı değiştir → `PATCH` · Bir nesneyi rotaya iletmek → `Depends` · Gerçek sunucu olmadan test → `TestClient`